Bookly AI Appointment Agent

Integration Audit & Frontend Handover

Complete technical documentation covering REST APIs, Socket.IO events, conversation flows, state management, MongoDB schema, external integrations, AI agents, TypeScript contracts, and testing checklist.

Generated: June 2026 | Audience: Frontend Team

1. System Architecture

Stack: FastAPI + Socket.IO ASGI | LangGraph orchestration | OpenAI LLM + Embeddings | MongoDB Atlas | ChromaDB | Next.js frontend

Frontend Layer (port 3000)

- Next.js - ChatPanel.tsx uses REST POST /chat
- Socket.IO client - user_message, bot_response, salon_cards, typing_start/end
- Standalone index.html - socket-only alternative UI
- localStorage - conversation persistence across refreshes

Backend Layer (port 8000)

- FastAPI - 9 REST endpoints
- CORS: allow_origins=['*']
- Socket.IO ASGI mounted at /socket.io
- Lazy dependency init on first request

LangGraph Workflow Nodes

- detect_intent -> route_intent
- -> faq_policy (RAG flow)
- -> service_prepare -> service_check -> location_check -> discover
- -> booking (date/time extraction + confirmation)
- -> general (fallback / greeting)

Data Source Priority

- 1. MongoDB Atlas (bookly) - vendor users.businessAddress
- 2. Categories API - localhost:4006/user/allCategories
- 3. mock_salons.json - fallback when MongoDB returns 0
- 4. ChromaDB - FAQ + policy knowledge base (.chroma/)

Request Flow

```
Frontend -> POST /chat OR socket user_message
OrchestratorService.chat() / socketio_server.user_message()
ensure_user_context(session_id, user_id) [MongoDB users lookup, cached per session]
OrchestratorWorkflow.run(GraphState) [LangGraph nodes]
detect_intent [OpenAI] -> route -> nodes -> reply + recommendations
ChatResponse { session_id, reply, intent, structured, recommendations }
[Socket only] emit bot_response -> emit salon_cards? -> emit typing_end
```

Known Gaps (Before Production)

[HIGH] BookingService is in-memory only: Bookings lost on restart; MongoDB bookings not wired.

[HIGH] No availability check: slots collection never queried before confirming booking.

[MED] No authentication: All APIs open. Plan auth headers before production.

[MED] SALON_INFO_API_URL unused: Declared in config but never called.

[LOW] mock_salons.json UAE bias: Non-UAE locations with no MongoDB data return mock UAE salons.

2. REST API Inventory

Base URL: http://localhost:8000 | Content-Type: application/json | No authentication required

POST /chat [Core]

Send user message; runs full LangGraph workflow.

```
Request:  { session_id, message, user_id? }
```

```
Response: { session_id, reply, intent, structured, recommendations[] }
```

Errors: 503 Service not initialized

POST /session/start [Core]

Load MongoDB user profile; return personalized greeting.

```
Request:  { session_id, user_id? }
```

```
Response: { session_id, greeting, context: SessionContext }
```

Errors: 503 Service not initialized

GET /health [Infra]

Liveness probe - RAG + category cache status.

```
Request:  -
```

```
Response: { status, app, rag_ready, categories_cached }
```

GET /categories [Data]

All service categories (TTL 1h cache).

```
Request:  -
```

```
Response: Category[] { id, name }
```

GET /debug/salons [Debug]

MongoDB discovery probe (diagnostic).

```
Request:  Query: location?, category?, service?
```

```
Response: { query, diagnosis, count, matches[] }
```

POST /book-appointment [Booking]

Direct booking (bypasses chat).

```
Request:  { session_id, salon?, service?, date?, time? }
```

```
Response: { booking_id, status: confirmed, payload }
```

Errors: 400 Missing fields | 503

POST /debug-rag [Debug]

Inspect RAG pipeline.

```
Request:  { question }
```

```
Response: { question, retrieved_chunks[], answer, explanation }
```

Errors: 503

POST /debug-intent [Debug]

Test intent classification.

```
Request:  { message }
```

```
Response: { message, intent, category, service, explanation }
```

Errors: 503

POST /upload [Utility]

Upload .docx knowledge document.

```
Request:  multipart file: *.docx
```

```
Response: { status, filename }
```

Errors: 400 Only .docx

3. Socket.IO Event Inventory

Path: /socket.io | Transport: async WebSocket | CORS: all origins

connect (S->C)

Trigger: Client connects

```
Payload: { session_id, reply }
```

Notes: Emits welcome greeting immediately.

disconnect (C->S)

Trigger: Client disconnects

```
Payload: -
```

Notes: Logged server-side only.

user_message (C->S)

Trigger: User sends message

```
Payload: { session_id, message, user_id? }
```

Notes: Triggers typing_start -> workflow -> bot_response -> salon_cards? -> typing_end

typing_start (S->C)

Trigger: After user_message received

```
Payload: { session_id }
```

Notes: Show typing indicator.

bot_response (S->C)

Trigger: After workflow completes

```
Payload: { session_id, reply, intent?, structured? }
```

Notes: Always emitted. structured.booking_payload on booking steps.

salon_cards (S->C)

Trigger: When recommendations exist

```
Payload: { salons: [{ salonId, salonName, rating, distance, location, services[], reason, redirectUrl }] }
```

Notes: Only when salons found. Frontend renders clickable cards.

typing_end (S->C)

Trigger: After all events emitted

```
Payload: { session_id }
```

Notes: Hide typing indicator.

4. Conversation Flow Analysis

Greeting / General

1. User sends hello or vague message
2. IntentAgent -> general_conversation
3. route_intent -> general node
4. Check SessionContext for fullName (from MongoDB)
5. Reply: Welcome back {name} OR generic welcome

FAQ / Policy (RAG)

1. User asks knowledge question (e.g. What is Bookly?)
2. IntentAgent -> faq | policy
3. route_intent -> faq_policy node
4. FaqAgent.answer() -> RagStore.retrieve_with_scores(top_k=8)
5. GPT-4o-mini synthesises answer from retrieved chunks
6. Reply: RAG-grounded answer only

Salon Search (location only)

1. User: Show salons in Noida
2. IntentAgent -> salon_search (no service, no category)
3. service_prepare: salon_search=true; location extracted
4. service_check: skipped
5. discover: MongoDB users.businessAddress query
6. ranking_agent.rank() -> top-5 cards
7. Reply + salon_cards event

Service Search -> Location -> Booking

1. User: Hair Smoothing
2. IntentAgent -> service_search; category=Hair
3. location_check: ask preferred location
4. User: Noida -> discover MongoDB Hair+Noida
5. BookingDraft auto-set; stage=awaiting_date
6. User: date -> stage=awaiting_time
7. User: time -> BookingService.create_booking()
8. Reply: Confirmed + booking_id (no name/phone requested)

Service + Location in One Message

1. User: Hair Smoothing in Noida
2. service + location extracted in same turn

3. discover runs immediately - no location prompt

Appointment Cancellation

1. User: Cancel my appointment
2. IntentAgent -> appointment_cancellation
3. route_intent -> faq_policy (policy explanation)

5. State Management

All state is server-side in ConversationStore (in-memory, keyed by session_id). GraphState (TypedDict) is the per-request envelope. No client-side session cookies.

ConversationStore Fields

```
_messages: dict[session_id -> Message[]]  last 12 as LLM history
_booking_drafts: dict[session_id -> BookingPayload]
_booking_stages: dict[session_id -> awaiting_date | awaiting_time]
_active_booking_sessions: set[session_id]
_user_contexts: dict[session_id -> SessionContext]  loaded once from MongoDB
_salon_lookup: dict[session_id -> {name: vendor_id}]
```

GraphState Fields

Field	Type	Lifecycle	Notes
session_id	string	Per request	UUID from frontend
intent_analysis	IntentAnalysis	detect_intent	intent+service+category+location
salon_search	boolean	service_prepare	Skip service requirement
service	string|null	service_prepare	Null for salon_search
category	string|null	service_prepare	Never defaulted
location	string	service_prepare	Free-form city or UAE area
recommendations	SalonRec[]	discover	Max 5 cards
booking_payload	BookingPayload	booking	salon_id, date, time, user_id
booking_stage	string	discover/booking	awaiting_date|awaiting_time
UserContext	SessionContext	ensure_user_context	Cached per session

6. MongoDB Database Analysis

Database: bookly. IMPORTANT: "addresses" collection stores CUSTOMER addresses, NOT vendor locations. Vendor location = users.businessAddress.

users

Purpose: Vendors AND customers (userType field)

Used by: SalonRepository, UserService

services

Purpose: Individual vendor services

Used by: SalonRepository (category+service search)

categories

Purpose: Service category taxonomy

Used by: SalonRepository, CategoryService

reviews

Purpose: Customer ratings for vendors

Used by: SalonRepository (avg rating)

addresses

Purpose: Customer saved addresses (NOT salon locations)

Used by: Supplementary city search

bookings

Purpose: Native app appointments [NOT INTEGRATED]

Used by: Not wired to chatbot

slots

Purpose: Vendor availability [NOT INTEGRATED]

Used by: No availability check before booking

Key Relationships

```
users._id <- services.vendorId
users._id <- reviews.vendorId
categories._id <- services.categoryId
users._id <- addresses.userId (customers only)
```

7. External Integrations & Environment Variables

Variable	Description
OPENAI_API_KEY	Required for LLM + RAG
OPENAI_MODEL	gpt-4o-mini
OPENAI_EMBEDDING_MODEL	text-embedding-3-small
MONGODB_URI	Atlas connection string
MONGODB_DB_NAME	bookly
CATEGORIES_API_URL	http://localhost:4006/user/allCategories
SALON_INFO_API_URL	https://admin.bookly.ae:2053/user/viewVendor (unused)
RAG_PERSIST_DIRECTORY	.chroma
RAG_COLLECTION_NAME	bookly_knowledge
TOP_K_RETRIEVAL	8
CATEGORIES_CACHE_TTL_SECONDS	3600
HOST / PORT	127.0.0.1 / 8000

Integration Summary

OpenAI GPT-4o-mini

IntentAgent structured output + FaqAgent synthesis

OpenAI Embeddings

RagStore -> ChromaDB upsert/query (top_k=8)

MongoDB Atlas

SalonRepository 4-query batched O(1) fetch; UserService profile load

ChromaDB

Local .chroma/ vector store for FAQ + policy docs

Categories API

GET /allCategories, TTL 1h, 8-category fallback

8. AI Agent Analysis

IntentAgent (intent_agent.py)

Classify into 9 intents. Heuristic + GPT structured output.

```
Output: IntentAnalysis { intent, service, category, location }
```

CategoryAgent (category_agent.py)

Map service -> category via keyword match.

```
Output: string | null
```

FaqAgent (faq_agent.py)

RAG FAQ/policy answers from Bookly knowledge base only.

```
Output: string (answer)
```

LocationAgent (location_agent.py)

Normalise location + search radius (UAE 10km, else 15km).

```
Output: LocationAnalysis
```

SalonDiscoveryAgent (salon_discovery_agent.py)

Bridge to SalonService.discover() -> MongoDB.

```
Output: list[Salon]
```

SalonRankingAgent (salon_ranking_agent.py)

Score: 40% service, 30% distance, 20% rating, 10% avail.

```
Output: list[SalonRecommendation] top 5
```

BookingAgent (booking_agent.py)

Extract date/time from free-text. No name/phone requested.

```
Output: BookingPayload
```

Intent Routing Matrix

Intent	Node	Action
faq / policy	faq_policy	FaqAgent -> ChromaDB -> OpenAI
salon_search	service_prepare -> discover	Location-only MongoDB query
service_search	service_prepare -> location -> discover	Service+category+location
appointment_booking	booking	Extract date/time, confirm
appointment_cancellation	faq_policy	Policy explanation via RAG
general_conversation	general	Personalised or generic greeting

9. TypeScript Interface Contracts

Paste into types/bookly.d.ts or equivalent.

```
interface ChatRequest { session_id: string; message: string; user_id?: string; }
interface SessionStartRequest { session_id: string; user_id?: string; }

interface ChatResponse {
  session_id: string; reply: string; intent: IntentType;
  structured: { intent, category, service, location, booking_payload, user_context };
  recommendations: SalonRecommendation[];
}

type IntentType = "faq"|"policy"|"salon_search"|"location_search"|"service_search"|
  "salon_discovery"|"appointment_booking"|"appointment_cancellation"|"general_conversation";

interface SalonCard {
  salonId: string; salonName: string; rating: number|null;
  distance: string|null; location: string|null;
  services: string[]; reason: string|null; redirectUrl: string|null;
}

interface BookingPayload {
  salon?: string; salon_id?: string; service?: string; category?: string;
  date?: string; time?: string; user_id?: string;
}

interface SessionContext {
  user_id, full_name, email, phone_number, gender, nationality, business_location;
}

// Socket: UserMessageEvent (C->S), BotResponseEvent (S->C),
//         SalonCardsEvent (S->C), TypingEvent (S->C)
```

10. Integration Testing Checklist

Priority	Area	Test
P0	Backend	GET /health returns status ok, rag_ready, categories_cached
P0	Backend	POST /chat returns 200 with reply and intent
P0	Backend	Show salons in Noida -> salon_search + recommendations
P0	Backend	Hair Smoothing in Noida -> service, category, cards
P0	Backend	What is Bookly? -> faq RAG answer
P0	Backend	Full booking flow -> booking_id in structured
P0	Socket	connect emits bot_response with welcome
P0	Socket	user_message emits salon_cards when recommendations exist
P0	MongoDB	GET /debug/salons?location=Noida count > 0
P0	Categories	GET /categories returns >= 8 items
P0	Frontend	salon_cards populates card UI with salonName, rating, redire
P1	Backend	POST /session/start with user_id returns greeting with fullN
P1	Socket	typing_start before bot_response; typing_end after
P1	RAG	POST /debug-rag cancellation policy returns chunks
P2	Frontend	localStorage persists conversation on refresh

11. Frontend Integration Guide

Chat Screen (ChatPanel.tsx)

Primary integration: REST POST /chat (not Socket in current Next.js UI)

```
POST /chat
Body: { session_id: uuid, message: string, user_id?: string }
Response: { reply, intent, structured, recommendations[] }
```

Loading: set loading=true before fetch, false after response.

Success: display reply; if recommendations.length > 0 render salon cards.

Error: catch HTTP 503 or network error; show user-friendly message.

Socket Integration (index.html / production)

```
Connect: io('http://localhost:8000')
Emit: socket.emit('user_message', { session_id, message, user_id })
Listen: bot_response, salon_cards, typing_start, typing_end
```

Session Start (optional)

```
POST /session/start
Body: { session_id, user_id }
Response: { greeting, context: { full_name, email, ... } }
```